

Module 2

# Authentication

# What is Authentication?

# The basis of computer security is controlled access



## Authentication

The process of ascertaining or confirming an identity (verifying the **someone** before granting access).

# AuthN is critical for security of systems

## Authorization (AuthZ)

Permissions, capabilities, and access control are all contingent upon knowing the identity of the subject.



## Username & Secret

Typically parameterized as a **username** and a **secret**.

- The secret attempts to limit unauthorized access.
- Desirable properties include being **unforgeable**, **unguessable**, and **revocable**.

# Identification vs. Authentication



## Identification

Asserting who a person is.

Digital identity properties:

- Well known
- Predictable
- Guessable

### Examples

Email, phone number, bank account, username, etc.



## Authentication

Proving an asserted identity.

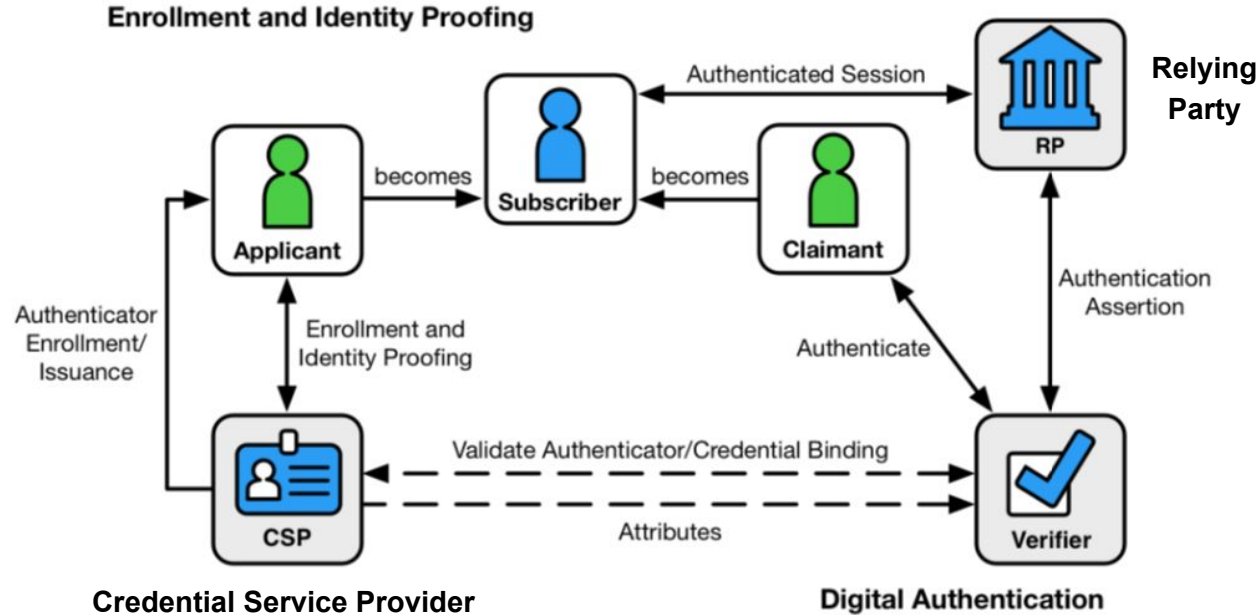
Core verification basis:

- Based on private item(s)
- Confirms the person is who they say they are

### Examples

Passwords, security questions, face, fingerprints, cards, tokens, etc.

# Framework: NIST 800-63-3



# Authenticators (= Types of “secrets”)



## Something you **know**

Memorized information such as passwords, PINs, passphrases, or security questions.



## Something you **have**

Physical objects like smart cards, hardware tokens, key fobs, or mobile devices.



## Something you **are**

Inherent physical traits including fingerprints, facial recognition, or iris patterns.

## One vs. **multiple** factors of authentication (MFA)

Multi-Factor Authentication (MFA) drastically improves security by combining two or more independent factors.

# Authenticators



**Memorized  
secrets**



**Look-up  
secrets**



**Out-of-band  
devices**



**Single-factor  
OTP device**



**Multi-factor  
OTP device**



**Single-factor  
crypto software**



**Single-factor  
crypto device**



**Multi-factor  
crypto software**

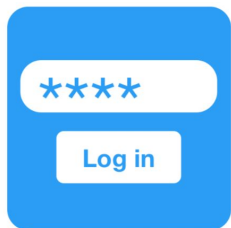


**Multi-factor  
crypto devices**



# Passwords

# Memorized secrets



## Something You Know

The classic, most widely implemented factor of entity authentication.

### What are they?

Also known as Passwords (or PINs when purely numeric).

### The Strength Principle

Must be "strong" enough to make guessing impractical.

### The Usability Paradox

Thought to be fairly straightforward to use. **Are they really?**

# Passwords: The good... and the ugly!

Features of passwords:

- Require **no training**: everyone knows passwords!
- **Cheap** to implement: can be implemented without a physical device.
- **Convenient**: users can authenticate from without carrying anything extra.
- **Easy to manage**: passwords can be centrally managed.
- **Widely supported**: every operating system, website, application, supports passwords.

However, given what we know about human memory:

- **People can't remember** infrequently-used or infrequently changed items
- We **can't forget** on demand
- **Recall** is harder than **recognition**
- **Non-meaningful words** are more difficult to remember.

# Password Storage: Plain Text

## Storage Requirement

**Validation:** System must validate passwords provided by users.

**Storage:** Therefore, passwords must be stored somewhere.

**Basic approach:** Storing as plain text.

| username | password   |
|----------|------------|
| alice    | p4ssw0rd   |
| bob      | ilovealice |
| charlie  | 123456     |

# Password Storage: Plain Text

## Attack Scenario

### 01. System Compromise

Attackers breach network boundaries and compromise core systems.

### 02. File Theft

They search for and successfully steal the system password file.

### 03. Immediate Exploitation

If stored in plain text, passwords are ready to use. Attackers can immediately log-in as any user, **including root!**



Critical Security Standard

**Passwords should never be stored in plain text.**

# Password Storage: Hashed Passwords

## Core Idea

Store a cryptographically hashed version of the password instead of plain text, using a one-way function.



How does the system work now?

## Cryptographic Properties

### 01. Deterministic

The same password always produces the exact same hash:  $\text{hash}(x) = \text{hash}(x)$

### 02. High Entropy

A tiny change in input yields a completely different output:

`MD5('security') = e91e6348157868de...`

`MD5('Security') = 2fae32629d4ef4fc...`

### 03. Collision Resistant

It is computationally infeasible to find a  $y$  such that  $\text{hash}(x) = \text{hash}(y)$ .

# Problem: Weak Passwords

## Vulnerability of Hashes

Are they secure from cracking?

**No!**

### Core Challenge

While the hash function is hard to invert, hashed passwords are still vulnerable to offline cracking.

### Attack Strategies

Different attack strategies exist based on a fundamental fact: **exploiting predictable password choices**.



The Human Element

**Reports since the late 1970s repeatedly suggest that people choose weak passwords.**

# Problem: Weak Passwords

## Morris & Thomson (1979)

Early study on UNIX password security

| er | Percentage | Structure                               |
|----|------------|-----------------------------------------|
|    | <1%        | Single ASCII character                  |
|    | 2%         | Two ASCII characters                    |
|    | 14%        | Three ASCII characters                  |
|    | 14%        | Four alphabetic letters                 |
|    | 21%        | Five alphabetic letters, all the same   |
|    | 18%        | Six lowercase alphabetic letters        |
|    | 15%        | Words in dictionaries or lists of names |
|    | 86%        | Total of all categories above           |

## Egg Online Bank (2002)

Analysis of customer-chosen PINs

**50%**

Family member names used

**8%**

Celebrity names used

## Imperva (2009)

Analysis of leaked credentials

**34M**

Facebook passwords analyzed

**30%**

Shorter than 7 characters

**50%**

Trivial (names or dictionary words)



# Cracking Passwords: Brute Force

## Short Passwords

Length 3 or less (not case-sensitive)

**18,278**

Total Combinations  
( $26^1 + 26^2 + 26^3$ )

At 1 ms per attempt:

**~5 Hours**

(~300 minutes total time)

## The Scale Limit

Why brute force gets harder

### Exponential Growth

What if passwords are longer or  
have more base symbols?

**CRITICAL FACTOR**

But search can be parallelized!

## 6-Char Passwords

~690 Billion combinations

x86 Server

**3.5 Hours**

Using standard server cpu

Modern GPU

**15 Minutes**

Through parallel hardware

# Cracking Passwords: Brute Force

## Simple Brute Force

### Short Passwords

Length 3 or less (not case-sensitive)

**18,278** 

 Total Combinations  
( $26^1 + 26^2 + 26^3$ )

At 1 ms per attempt:

**~5 Hours**   
(~300 minutes total time)



## The Challenge

### The Scale Limit

Why brute force gets harder

**Exponential Growth.**

What if passwords are longer or  
have more base symbols?



**CRITICAL FACTOR**

But search can be parallelized!

## Accelerated Attacks

### 6-Char Passwords

~690 Billion combinations

x86 Server

**3.5 Hours**

Using standard server CPU



Modern GPU

**15 Minutes**

Through parallel hardware



# The EFF DES Cracker (1998)

## Encryption Strength

- DES uses a relatively small **56-bit key**.
- Similar practical attacks have been demonstrated against **MD5** and **SHA1**.

**\$250K**

### Budget

Total cost in 1998 to  
build

**56h**

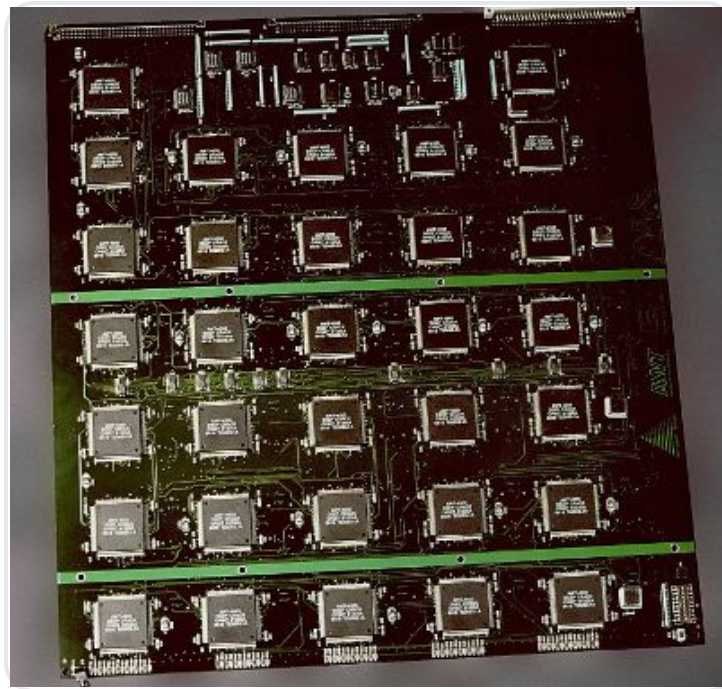
### Crack Time

To brute-force all  
keys

**1,856**

### ASIC Chips

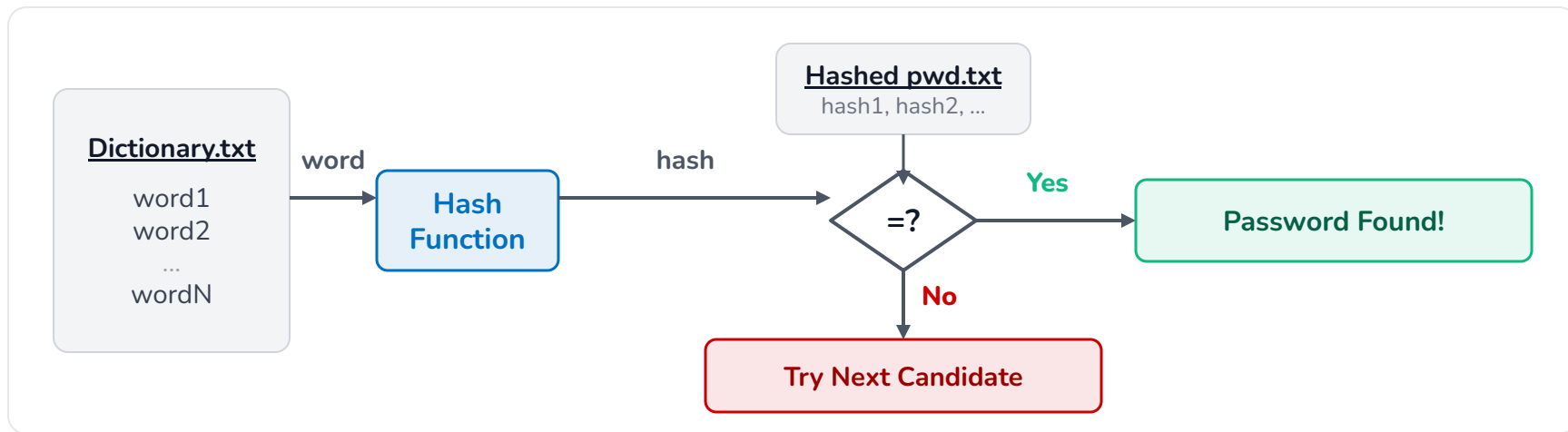
Custom parallel  
processors



# Cracking Passwords: Dictionary Attacks

## Premise

- Because people do not choose truly random passwords, why not take a pre-compiled list of candidates (a **dictionary**) and systematically test them?
- **Result: It is common for 60-70% of hashed passwords to be cracked in less than 24 hours.**



# Rainbow Tables

## 1. Deterministic Hashes

- Cryptographic hash functions are deterministic.
- The **same input** always produces the **same output**.
- This consistency forms the mathematical foundation for pre-computation attacks.

## 2. Offline Strategy

- Adversaries build pre-computed lists of hashes **offline**.
- What is the advantage?
- It trades offline preparation time to achieve near-instantaneous online cracking.

## 3. Rainbow Tables

- Contain a structured spectrum of passwords up to length N.
- Highly compact, optimized, and effective.
- Enables time-space efficient reversal of hashes via **TMTD attacks**.

# Rainbow Tables

## 1. Deterministic Hashes



Cryptographic hash functions are deterministic.



The **same input** always produces the **same output**.



This consistency forms the mathematical foundation for pre-computation attacks.

## 2. Offline Strategy



Adversaries build pre-computed lists of hashes **offline**.



What is the advantage?



It trades offline preparation time to achieve near-instantaneous online cracking.

## 3. Rainbow Tables



Contain a structured spectrum of passwords up to length N.



Highly compact, optimized, and effective.



Enables time-space efficient reversal of hashes via **TMTO attacks**.

# Hardening Password Hashes

## 1. The Salt Solution

- Make each password hash **uniquely identifiable**.
- Achieved by adding a random number (**salt**) to each password before hashing it.

## 2. Rationale

- $\text{Hash}(p + r1) \neq \text{Hash}(p + r2)$
- Every user is assigned a completely **unique and random** salt.
- Two users with identical passwords will produce **different hashes**.

## 3. Public Storage

- The salt is stored in **plain text** directly alongside the hash.
- It is **not a secret!**
- *Think about it: the goal is uniqueness of hashes, not obscurity of the salt.*

# Breaking and Fixing Hashed Passwords

## 1. Best Practice

- Stored passwords should **always be hashed** to protect user credentials.

## 2. The Threat

- Increasingly faster **specialized hardware (GPUs)** with 100s/1000s of cores.
- Can be easily **rented from cloud providers** to launch high-speed attacks.

## 3. The Challenge

- **How can we defend** against this raw computing power?



# Breaking and Fixing Hashed Passwords

## 1. Key Idea

- **Deliberately Slow:** make the hashing process slower.
- Cryptographic hashes are designed for speed, which is counterproductive for passwords.

## 2. Key Stretching

- **Multiple Rounds:** Hash the password multiple times recursively.
- For example, **bcrypt** traditionally uses 25 rounds of DES.

## 3. Adaptive Hashes

- **Work Factor:** Use special hash functions with configurable slowness.
- Industry standards include **PBKDF2**, **scrypt**, and bcrypt.

# Choosing Passwords

# What is a strong password?

Metric: **Entropy**

$$S = \log_2 N^L$$

Where:

- **N** = number of possible symbols
- **L** = length of the password
- **S** = password "strength" (entropy) in bits

## Application & Limits

- It calculated the length (**L**) needed to achieve a target security strength (**S**).
- **Crucial Assumption:** This metric only holds true for a randomly generated password!

### Is this useful in practice?

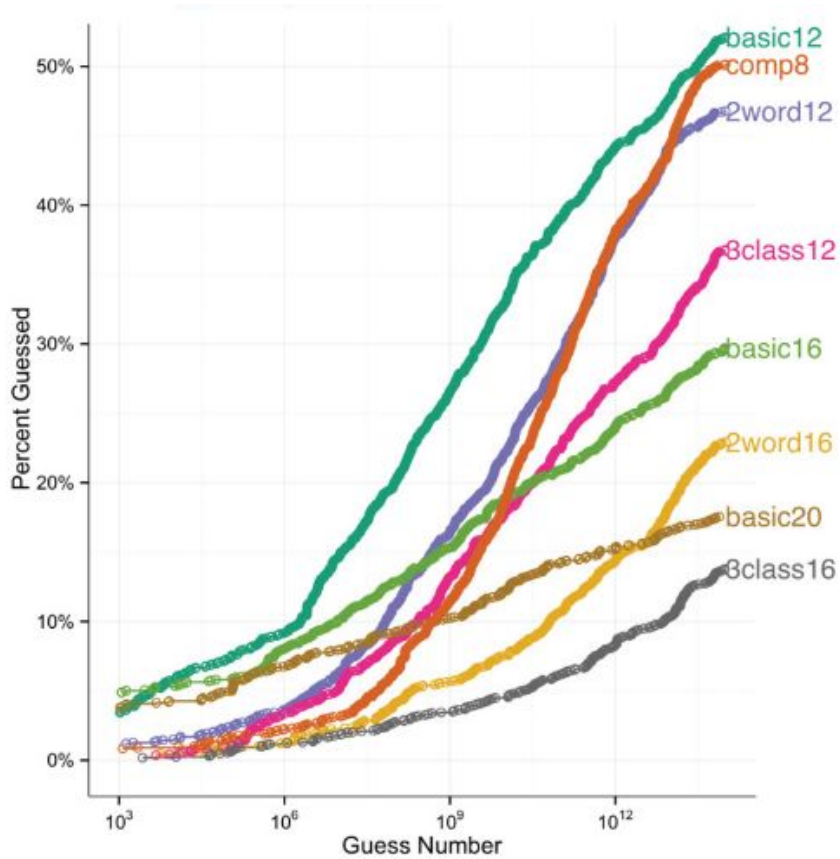
Since humans rarely choose truly random passwords, traditional entropy metrics often fail in real-world scenarios.

# Choosing passwords: Mental Algorithms

- Popular after years of security training
- Sold as “secure” for years
- Variants of these procedures:
  1. Pick a word
  2. Capitalize the first or last letter
  3. Add a number (and maybe a symbol) to the beginning or end
- 1. Pick a word
  2. Replace some of the letters with symbols (s/a/@, s/s/\$, ...)
  3. Maybe capitalize the first or last letter (or both)

# Choosing passwords: Mental Algorithms

- Result is something like:
  - Computer3@
  - C14ssr00m
- But modern adversaries have sophisticated cracking tools:
  - No need for brute force cracking
  - Use dictionaries, including and passwords from prior leaks
  - Apply common rules found in mental algorithms



Shay et al. (ACM TISSEC 2016)

[https://www.blaseur.com/papers/tissec\\_1026.pdf](https://www.blaseur.com/papers/tissec_1026.pdf)

## Password Policies

### comp $n$ & basic $n$

Requires using at least  $n$  characters.

### $k$ word $n$

Combine at least  $k$  words using at least  $n$  characters.

### $d$ class $n$

Use at least  $d$  character types (upper, lower, digit, symbol) with at least  $n$  characters.

# Good heuristics for passwords

## The Entropy Formula

$$S = L \times \log_2 N$$

In determining password strength, **length (L)** matters more than the size of the **symbol pool (N)**.

## 1. Choose Longer Passwords

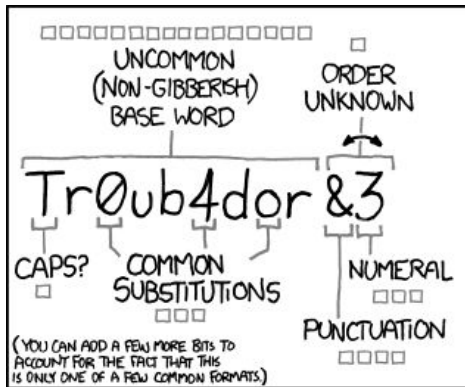
Aim for 16+ characters. Don't worry about forcing uppercase letters, digits, or complex symbols!

## 2. Use Mnemonics

Choose a memorable sentence or phrase and reduce it to the first letter of each word to create a high-strength, recallable secret.

## 3. Insert Random Variations

To further expand the search space, introduce a few random uppercase letters, digits, and symbols into your phrase at easy-to-remember spots.



~28 BITS OF ENTROPY

$2^{28} = 3 \text{ DAYS AT } 1000 \text{ GUESSES/SEC}$

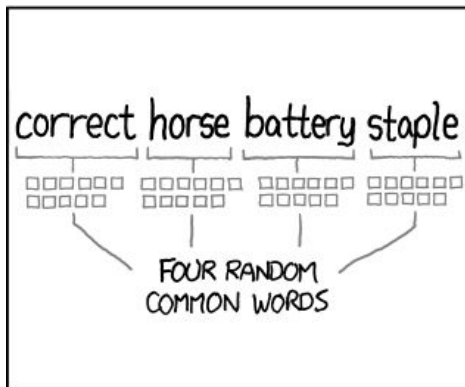
(PLAUSIBLE ATTACK ON A WEAK REMOTE WEB SERVICE. YES, CRACKING A STOLEN HASH IS FASTER, BUT IT'S NOT WHAT THE AVERAGE USER SHOULD WORRY ABOUT.)

DIFFICULTY TO GUESS: **EASY**

WAS IT TROMBONE? NO, TROUBADOR. AND ONE OF THE 0s WAS A ZERO?

AND THERE WAS SOME SYMBOL...

DIFFICULTY TO REMEMBER: **HARD**



~44 BITS OF ENTROPY

$2^{44} = 550 \text{ YEARS AT } 1000 \text{ GUESSES/SEC}$

DIFFICULTY TO GUESS: **HARD**

THAT'S A BATTERY STAPLE.

CORRECT!

DIFFICULTY TO REMEMBER: YOU'VE ALREADY MEMORIZED IT

THROUGH 20 YEARS OF EFFORT, WE'VE SUCCESSFULLY TRAINED EVERYONE TO USE PASSWORDS THAT ARE HARD FOR HUMANS TO REMEMBER, BUT EASY FOR COMPUTERS TO GUESS.

<https://www.xkcd.com/936/>



# Good verifiers

## 1. No Password Hints

Do not provide hints that could help attackers guess the secret.

## 2. Rate Limiting

Throttles or blocks repeated failed attempts to prevent brute-force attacks.

## 3. Permit "Paste" Functionality

Enables the use of modern password managers by allowing paste.

## 4. Visible Masking Options

Allows the user to see the password while typed to reduce errors.

## 5. No Forced Expiry Policies

Avoid imposing periodic password change policies that lead to weaker secrets.

## 6. Secure Salted Hashing

Store passwords salted and hashed using a high cost factor standard.

# Password Managers

# Password managers

Tool that allows users to manage and access passwords.

Stores all your passwords in a DB and protects it with a master password.

To retrieve a password (e.g., to access an online service), you type the master password to unlock the DB. In many cases the password manager will log in for you.

# Password managers

## Common features

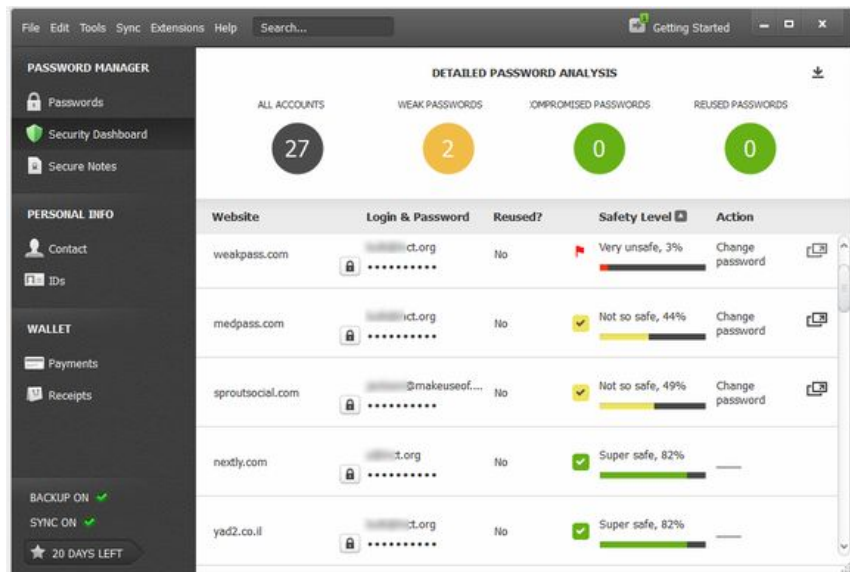
- Encrypted storage
- Password DB:
  - Local vs cloud
  - Synchronized among multiple devices
- Standalone vs integrated (e.g., with browsers)
- Password generation and expiration policies
- MFA to unlock it

# Password managers

## Extra features for product differentiation

- “Vaults”: Storage for sensitive info other than passwords, such as IDs, credit cards and payment info, pictures, docs, ...
- Alert service when visiting compromised sites
- Secure password sharing with someone else
- Travel mode (removes sensitive data from devices when crossing borders and restores it when you get back)

# Password managers



dashlane



1password

# Password managers

Some examples (go check them!)



# Password managers

Overall a good idea, right?

- ++ May improve security and compensate for weaknesses arising from having too many passwords.
- ++ Huge usability benefits in devices with small screens and/or multilayer keyboards (e.g., smartphones)
- Security: Single point of failure + are they really secure?
- Usability: Do they integrate seamlessly?



# Password managers

## Attacks: the cloud!



February 2020

Hello,

We are writing to inform you of a technical issue that affected the Google 'Download your data' service for Google Photos between November 21, 2019 and November 25, 2019, when it was fixed:

- Between November 21, 2019 and November 25, 2019, our records show you requested a Google 'Download your data' export, which included Google Photos content.
- Unfortunately, during this time, some videos in Google Photos were incorrectly exported to unrelated users' archives. One or more videos in your Google Photos account was affected by this issue.
- If you downloaded your data, it may be incomplete, and it may contain videos that are not yours.
- The underlying issue has been identified and resolved. We recommend you perform another export of your content and delete your prior export at this time.

We apologize for any inconvenience this may have caused. For further assistance, contact [Google Support](#). Please ensure you are signed in to the correct account when contacting support.



Whoa, what? @googlephotos?



February 2020

Hello,

We are writing to inform you of a technical issue that affected the Google 'Download your data' service for Google Photos between November 21, 2019 and November 25, 2019, when it was fixed:

- Between November 21, 2019 and November 25, 2019, our records show you requested a Google 'Download your data' export, which included Google Photos content.
- Unfortunately, during this time, some videos in Google Photos were incorrectly exported to unrelated users' archives. One or more videos in your Google Photos account was affected by this issue.
- If you downloaded your data, it may be incomplete, and it may contain videos that are not yours.
- The underlying issue has been identified and resolved. We recommend you perform another export of your content and delete your prior export at this time.

We apologize for any inconvenience this may have caused. For further assistance, contact [Google Support](#). Please ensure you are signed in to the correct account when contacting support.

3:51 AM - Feb 4, 2020 - Twitter Web App

2.6K Retweets 5K Likes



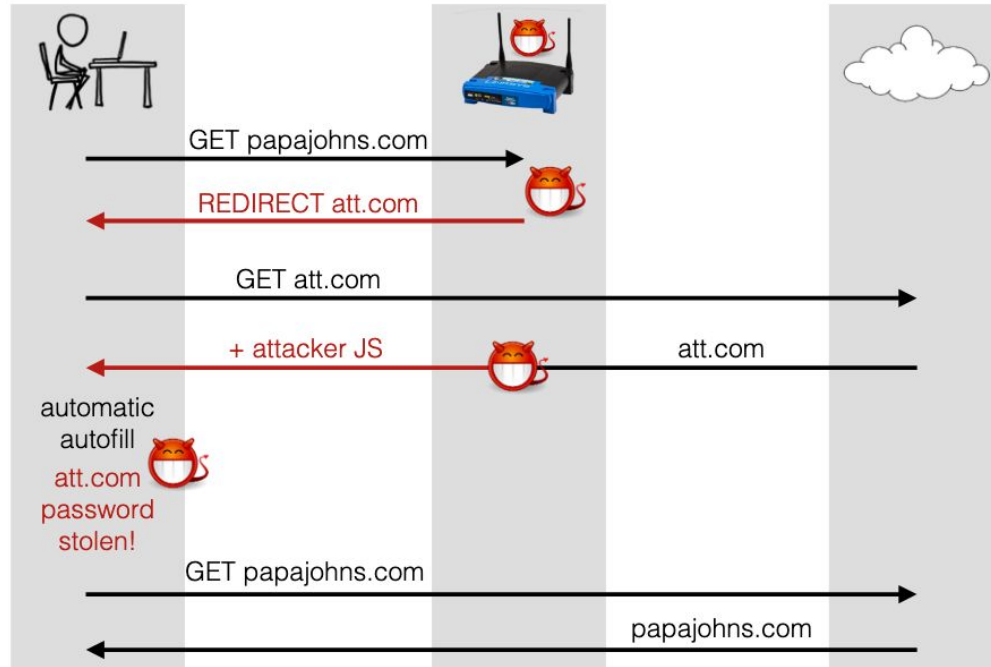
# Password managers

## Early attacks: the autofill attack (2014)

- Silver et al. “Password Managers: Attacks and Defenses”, USENIX Security Symposium 2014.
- Attacker extracts many (all) passwords from a PM if the user connects to a rogue WiFi network.
- All 10 popular PM found to be vulnerable.
- Threat model:
  - “Evil Coffee Shop Attacker”
  - Means temporary control of a router or access point

# Password managers

## Early attacks: the autofill attack (2014)



# Password managers

## Early attacks: the autofill attack (2014)

Autofill is one of the best usability features of PMs, and one that is difficult to implement securely.

After the attack, **LastPass** no longer autofills passwords in iFrames and **1Password** changed its policy for https pages over http conns.

# Password managers

## More attacks (2023)

**Bitwarden flaw can let hackers steal passwords using iframes**

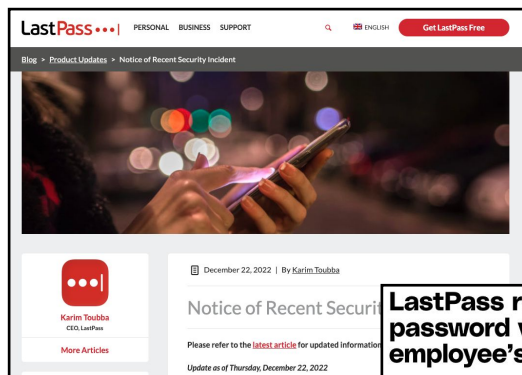
By **Bill Toulas**

March 8, 2023 05:08 PM 6

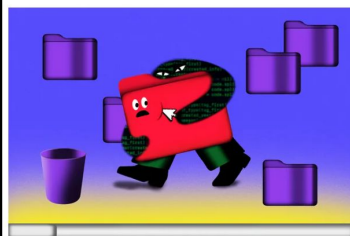


# Password managers

## More attacks: The LastPass 2022 Leak



### LastPass reveals attackers stole password vault data by hacking an employee's home computer



/ The password manager's latest update regarding two security breaches last year discloses how a threat actor accessed customer information.

By Jess Weatherhead, a news writer focused on creative industries, computing, and internet culture. Jess started her career at TechRadar, covering news and hardware reviews.  
Feb 28, 2023, 3:46 PM GMT+1 | 17 Comments / 17 Views

The attacker stole credentials from a senior DevOps engineer to gain access to shared cloud storage containing the encryption keys for customer vault backups.  
Illustration: Beatrice Bala

### The Catastrophic LastPass Breach Was Even Worse Than It Seemed

In December, the password-manager maker LastPass revealed that an August breach it had disclosed at the end of November was worse than the company originally thought, compromising encrypted copies of some users' password vaults, on top of other personal information. Now, the company has disclosed a second incident that began in mid-August and allowed attackers to rampage through the company's cloud storage and exfiltrate sensitive data. Attackers gained such extraordinary access by targeting a specific LastPass employee with deep system privileges

"This was accomplished by targeting [a] DevOps engineer's home computer and exploiting a vulnerable third-party media software package, which enabled remote code execution capability and allowed the threat actor to implant keylogger malware," LastPass wrote in an account of the situation. "The threat actor was able to capture the employee's master password as it was entered, after the employee authenticated with MFA, and gain access to the DevOps engineer's LastPass corporate vault."

To target the LastPass employee, attackers exploited a Plex Media Server software vulnerability that had already been long-patched at the time. The company issued a fix for the bug in May 2020, "roughly 75 versions ago," Plex said.

# Other Authenticators

# Look-up secrets



- Physical or electronic device that stores a shared secret
- Something you have
- Commonly used as "recovery keys" or 2FA

Secured by **Entrust**

|   | A  | B  | C  | D  | E  | F  | G  | H  | I  | J  |   |
|---|----|----|----|----|----|----|----|----|----|----|---|
| 1 | A3 | 11 | C5 | S3 | 57 | K1 | M9 | FQ | G7 | XD | 1 |
| 2 | T6 | O7 | Q5 | 29 | F3 | 1K | PE | CZ | N4 | K2 | 2 |
| 3 | 29 | ET | B7 | M3 | AZ | H7 | YJ | V8 | L0 | 48 | 3 |
| 4 | QK | XR | HR | U6 | 18 | N3 | AB | LU | 76 | X8 | 4 |
| 5 | P7 | D3 | 14 | LV | Y5 | G6 | Z9 | AW | 72 | S9 | 5 |

U.S. Patent 5,712,627

07824



# Look-up secrets



- Contents should be generated with an adequate RNG and each secret should have at least 20 bits of entropy
- Each secret in the token (e.g., a grid card) should be used only once

# Out-of-band devices



- Possessed and controlled by the user
- Can communicate securely with the verifier over a distinct (*secondary*) communication channel
- Can operate in different ways:
  - Transfer to 1<sup>st</sup> channel secret received via the 2<sup>nd</sup>
  - Transfer item observed in the 1<sup>st</sup> channel to the 2<sup>nd</sup>
- Limited if 2<sup>nd</sup> channel is PSTN

# Single-factor OTP device



- OTP = One Time Password
- Hardware or software with an embedded shared secret
- Does NOT require activation through another secret
- Displays OTP and the user enters it manually elsewhere
- Proves possession and control of the device
- Similar to look-up devices but secrets are cryptographically and independently generated by both parties.
- OTPs computed based on a nonce that is **time-based**, a **counter**, or a **challenge**.

# Single-factor OTP device



## Time-Based Token Authentication

Login: mcollings

Passcode: 2468159759

PASSCODE = PIN + TOKENCODE

Token code:  
Changes every  
60 seconds



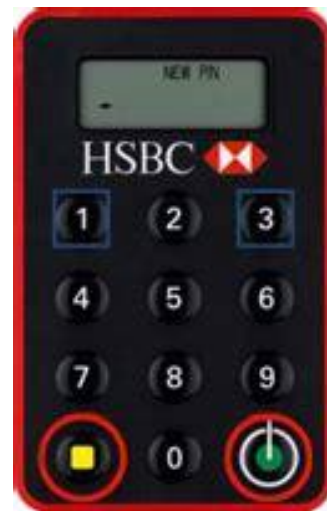
Clock  
synchronized to  
UCT

Unique seed

# Multi-factor OTP device



- Locked OTP device that needs to be unlocked through an additional FA, either something you know or something you are.



# Single-factor crypto software



- Cryptographic key stored on a disk or some “soft” media
- Authentication accomplished by proving possession of it
- Lots of protocols, though generally some form of signed message over a nonce.
- Stored in a suitably secure storage, e.g., keychain or TPM/TEE if available.

# Single-factor crypto devices



- Device with embedded crypto keys to perform crypto ops
- Provides authenticator output directly, often some form of signed message over a provided nonce
- Does not require a second FA to activate
- Key should not be exportable
- Should require physical activation (e.g., pressing a button)
- USB devices are common (e.g., YubiKey)

# Multi-factor crypto software



- Same as single-factor cryptographic software, but key access requires activation through a second FA, either something you know or something you are.
- Proves both possession and control of the key



# Multi-factor crypto devices



- Same as single-factor cryptographic device, but key access requires activation through a second FA, either something you know or something you are.
- Proves both possession of the device and control of the key

# Biometrics

Biological features based on some physical characteristic of the human body

fingerprint

hand geometry

retina and iris

voice

face, head

blood vessels in the hand

handwriting, signature

hand motion

# What's a good biometric feature?

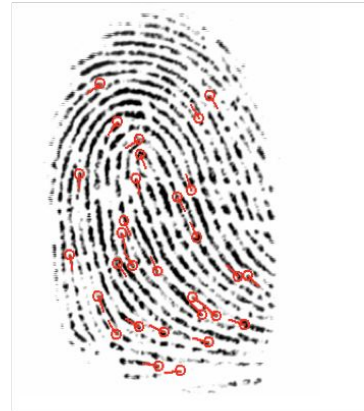
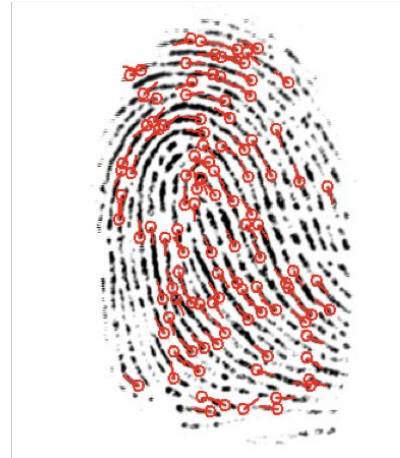
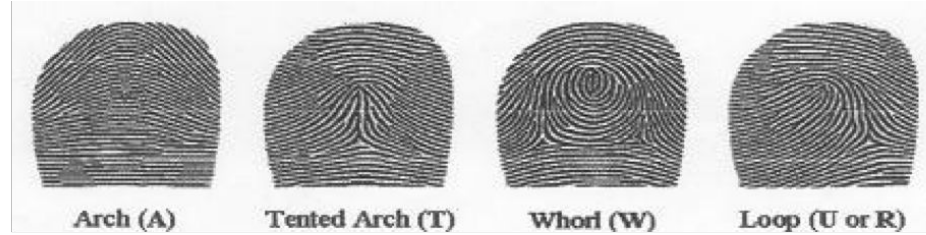
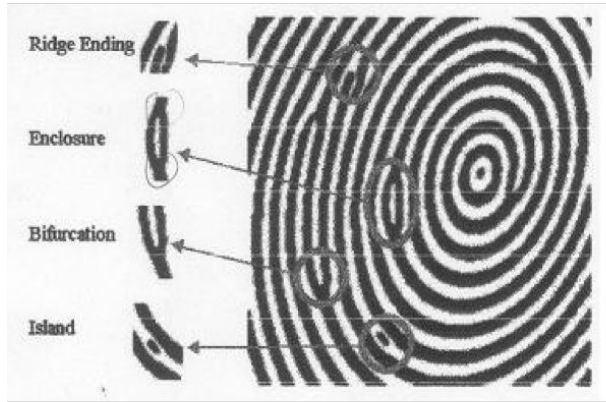
- 1) every person should have it (universality)
- 2) individuals can be distinguished by it (uniqueness)
- 3) should be invariant over time (permanence)
- 4) acquisition should be easy (collectability or measurability)
- 5) users should not have objections to its use (acceptability)
- 6) accuracy level and recognition speed must be adequate (performance)
- 7) should be difficult to counterfeit (resistance to circumvention)

# Enrolment and match

During **enrolment** the algorithm extracts a **pattern** (template) that represents a measurement of the biometric characteristic

During **authentication**, a current set of measurements is matched against the template. Authentication succeeds if both are “close enough”

# Fingerprints



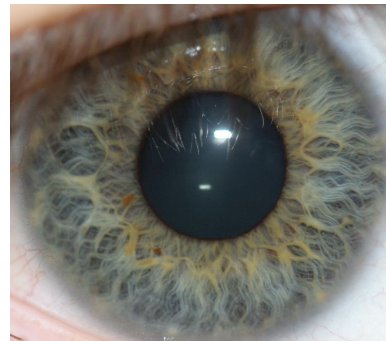
# Hand, Iris



Hand geometry  
reader



Hand vein reader



# Face / Head



Source: FT

[Inside China's surveillance state](#)

[https://www.youtube.com/watch?v=\\_yKga54tx6U](https://www.youtube.com/watch?v=_yKga54tx6U)

<https://www.youtube.com/watch?v=uReVvICTrCM>

<https://www.youtube.com/watch?v=ectdRsyj-zI>



# Threats to authenticators

- **Something you know** may be disclosed, guessed, observed or intercepted while entered, revealed by the user through phishing or social engineering
- **Something you have** may be lost, damaged, stolen, or cloned.
- **Something you are** may be replicated



# Authentication in Distributed Systems

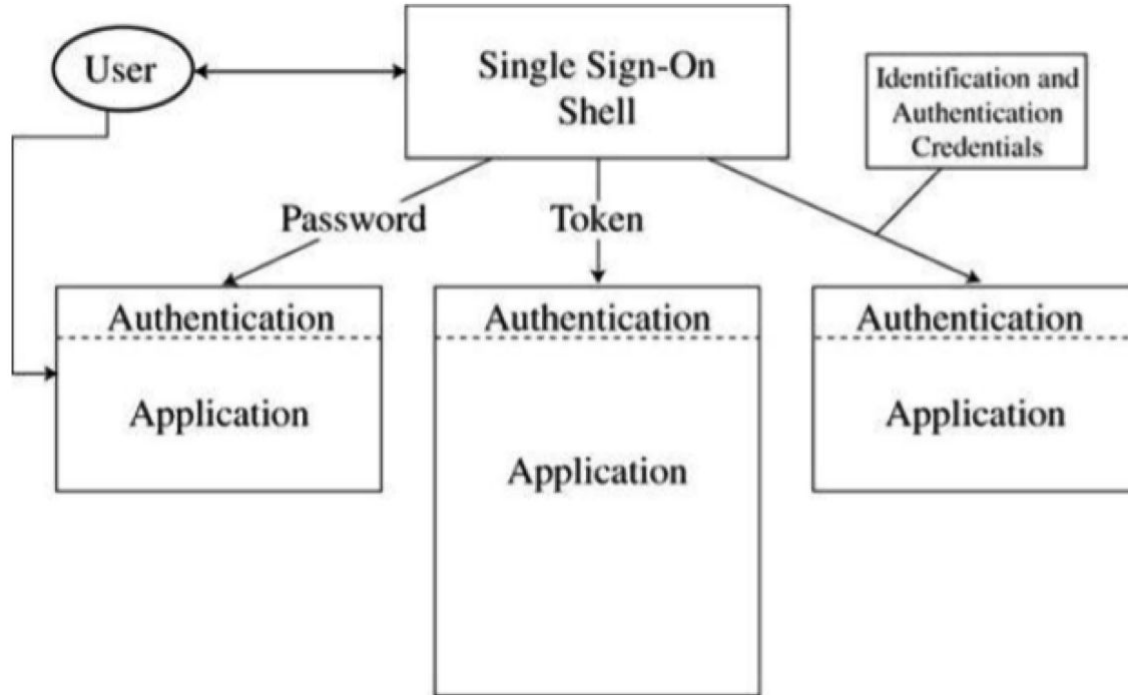
# Federated identity

Union of separate identification and authentication systems.

**Goal:** reduce user fatigue associated with managing multiple authenticators

- **Federated identity system:** One profile with one authentication method shared by a federation of apps and systems.
- **Single sign-on:** One identity management module replaces identification and authentication in all systems

# Single sign-on

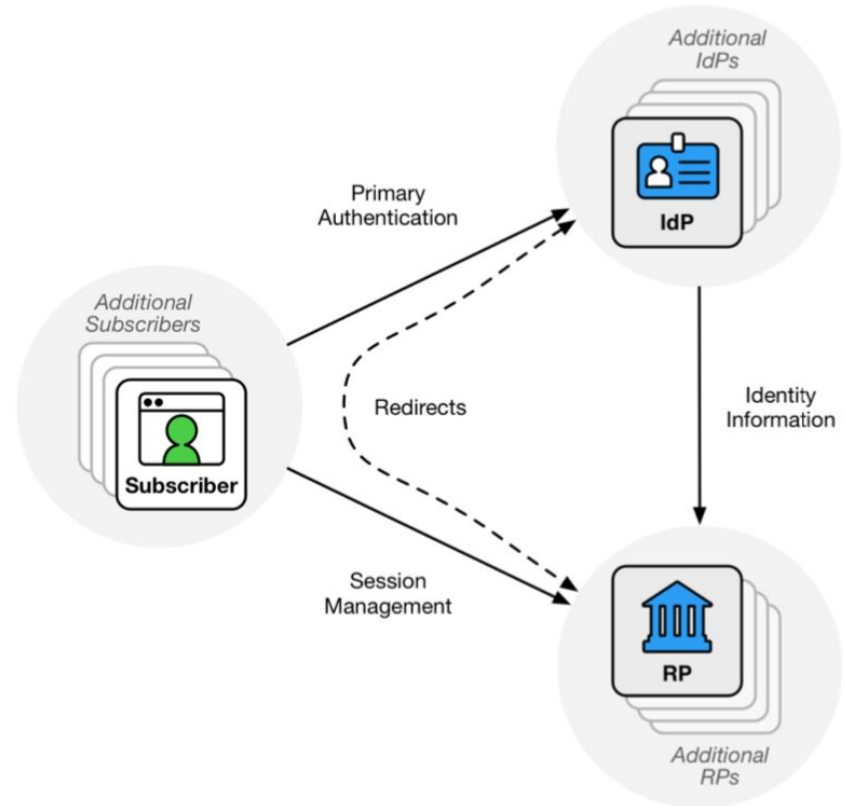


# Federations

**Subscriber** communicates both with the **IdP** (Identity Provider) and the **RP** (Relying Party), often through a browser.

IdP and RP communicate with each other through the **front channel** (using subscriber redirects) or the **back channel** (directly).

IdP **asserts** auth results and identity attributes.



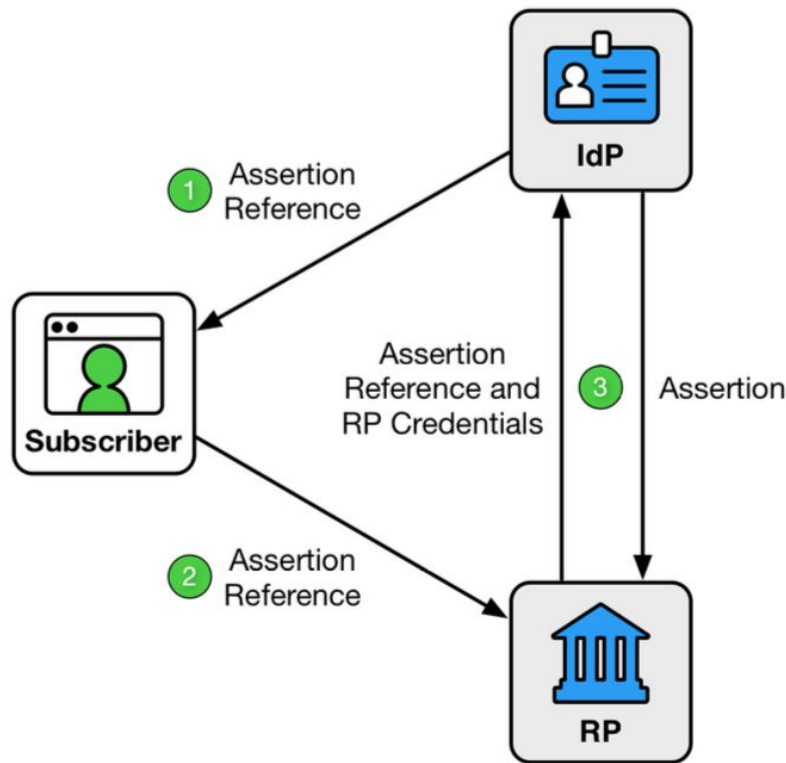
# Assertions

- Packaged set of attributes associated with an authenticated subscriber
- Passed from the IdP to the RP
- Primary use is authentication, but can be used for other purposes (e.g., personalization of a website).
- SHALL include:
  - Subject, issuer, audience, issuance, expiration, identifier, signature and authentication time.
  - May also include other attributes, e.g., info about the subscriber, device, location, etc.
- Sent over secure authenticated channels and protected:
  - Always signed by the IdP (PKC) or MACed with shared key

# Assertions presentation

## Back-channel presentation

1. IdP sends an assertion ref to the subscriber through the front channel.
2. Subscriber forwards the assertion ref to the RP through the front channel.
3. RP presents the assertion ref to the IdP through the back channel. IdP validates the credentials and return the assertion.



# Assertions presentation

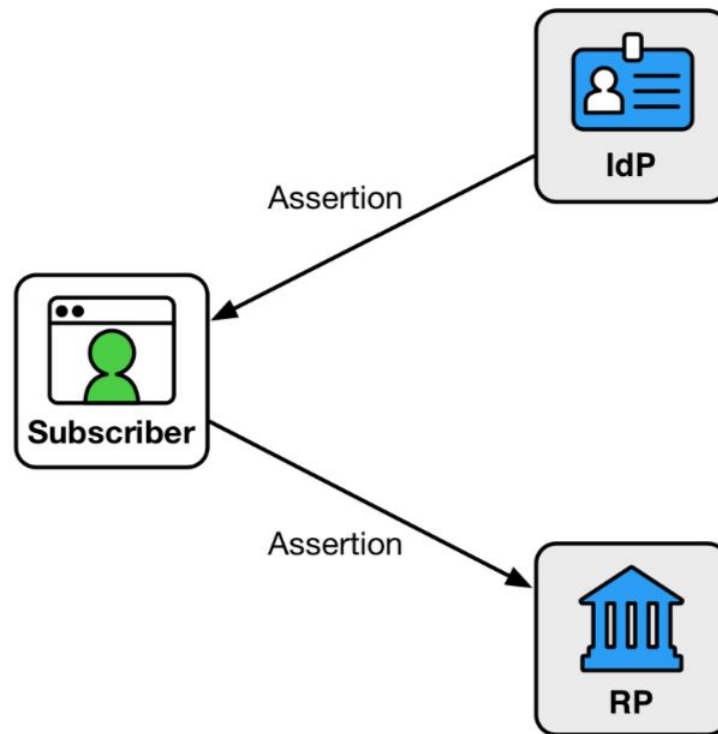
## Back-channel presentation

- More network transactions
- But information limited to only those parties that need it
- RP expects the assertion from the IdP, so the attack surface is reduced
- Assertion has to be properly validated:
  - Issuer verification
  - Signature verification
  - Time validation
  - Audience restriction

# Assertions presentation

## Front-channel presentation

1. IdP sends an assertion to the subscriber through the front channel.
2. Subscriber forwards the assertion to the RP through the front channel.





# Assertions presentation

## Front-channel presentation

- Less network transactions
- More difficult for the RP to query the IdP for additional attributes
- Assertion under the subscriber's control:
  - Visible to it, which could potentially cause leakages (e.g., user profiling) if redirected to unintended parties even if audience is restricted
- Assertion has to be properly validated:
  - Issuer verification
  - Signature verification
  - Time validation
  - Audience restriction

# Example: SAML

## Specific Assertion Markup Language

- XML-based framework for creating and exchanging authentication and attribute information over the Internet
- Three main components:
  - Assertions XML schema defines the structure of an assertion
  - SAML protocols used to request assertions and assertions refs
  - Bindings: underlying transport protocols (e.g., HTTP or SOAP)
- See [Wikipedia's SAML 2.0 page](#)

# Example: OpenID Connect (OIDC)

- Authentication layer built on top of OAuth 2.0 and JOSE (JSON Object Signing and Encryption)
- The IdP issues
  - An ID Token = signed assertion in the JWT (JSON Web Token) format.
  - A standard OAuth access token that can be used at the IdP to obtain a JSON with full info about the user
- The RP parses the ID Token and uses the OAuth access token

# Privacy requirements

- Federation involves transfer of personal attributes from a third party (the IdP) that is not otherwise involved in the transaction. This gives the IdP **visibility** into subscriber activities.
- CSPs often have business purposes different from identity provision.
- Back channel => reveals to the IdP where the subscriber is conducting a transactions
  - Multiple RP => **profiling & tracking**
- Trade-off with goals other than id proofing, such as fraud mitigation or to comply with legal process.

# Further reading: Trends

## Passwordless authentication

- Introduction:  
<https://auth0.com/blog/what-is-passwordless-authentication/>
- FIDO: <https://fidoalliance.org/fido2/>
- Google Passkeys: <https://developers.google.com/identity/passkeys>
- Apple Passkeys: <https://support.apple.com/en-us/102195>

## Continuous authentication

- <https://securityboulevard.com/2020/07/the-future-of-zero-trust-continuous-authentication/>